

- 面向对象核心思想 (e.g., 设计原则)
- 封装 (e.g., 目的、访问范围、重载、静态属性.....)
- 继承 (继承的优缺点、重写/覆盖、转型、super.....)
- 多态 (e.g., 多态的必要条件、表现形式.....)
- 设计原则&设计模式
- 其他内容：集合、异常、多线程、网络、I/O (各有1-2道选择题)

设计模式

单一职责原则：每个部分负责一个特定功能

开闭原则：对扩展开放，对修改关闭

接口隔离原则：对于不同功能使用多个接口，而不是统一接口

依赖倒置原则：面向接口编程而不是面向过程编程

里氏代换原则：使用父类的地方一定需要可以透明地使用子类

迪米特原则：软件实体应该尽量少地和其他实体发生作用

合成复用原则：多用组合聚合少用继承

设计模式复习

1. **原型（Prototype）模式**用一个已经创建的实例作为原型，通过**复制**该原型对象来创建一个和原型相同或相似的新对象。
2. **适配器模式**：的核心是将一个类的接口转换成客户希望的另外一个接口。
3. **装饰模式**：的核心是动态地给对象添加一些额外的职责。
4. **外观模式**：为多个复杂的子系统提供一个一致的接口，使这些子系统更加容易被访问
5. **策略模式**：定义一系列算法,把它们封装起来,并且使它们可相互替换。本模式使得算法可独立于使用它的客户而变化。
6. **访问者模式**：的核心是在不改变各个元素的类的前提下定义作用于这些元素的新操作。
7. **责任链模式**：把请求从链中的一个对象传到下一个对象，直到请求被响应为止。通过这种方式去除对象之间的耦合。
8. **观察者模式**：一对多关系，当一个对象发生改变时，把这种改变通知给其他多个对象，从而影响其他对象的行为。

创建型

单例模式

懒汉式 & 饿汉式

工厂模式

原理

用于封装对象的创建过程，通过一个**静态方法**根据输入参数返回不同类型的对象

例题

创建汽车

优点

- 可以使代码结构清晰，有效地封装变化
- 对调用者屏蔽具体的产品类
- 降低耦合度

适用场景

- 在任何需要生成**复杂对象**的地方
- 需要增加**依赖关系**时，可以降低对象之间的耦合度
- 需要系统又比较好的扩展性时，不同的产品用不同的实现工厂来组装

存在的问题

- 违背了**开闭原则**（可以使用**抽象工厂模式**解决）

原型模式

实现 `Cloneable` 接口之后实现 `clone()` 函数

返回 `super.clone()` 即可

优点

- 是内存二进制的拷贝，比 `new` 一个对象性能好很多
- 避开构造函数的约束

使用场景

- 资源初始化场景，类初始化需要消耗非常多的资源
- 性能和安全要求场景，产生对象需要繁琐的数据准备和访问权限
- 一个对象多个修改者

例题

```
interface Cloneable{
    public Object clone() throws CloneNotSupportedException;
}
```

```
class Realizetype implements Cloneable{
    Realizetype() {
        System.out.println("具体原型创建成功！");
    }
    public Object clone() throws CloneNotSupportedException {
        System.out.println("具体原型复制成功！");
        return (Realizetype)super.clone();
    }
}
```

具体原型创建成功！
具体原型复制成功！
`obj1==obj2?false`

```
public class PrototypeTest{
    public static void main(String[] args)throws CloneNotSupportedException {
        Realizetype obj1=new Realizetype();
        Realizetype obj2=(Realizetype)obj1.clone();
        System.out.println("obj1==obj2?"+(obj1==obj2));
    }
}
```



结构型

适配器模式

将一个类的接口转化为想要的接口

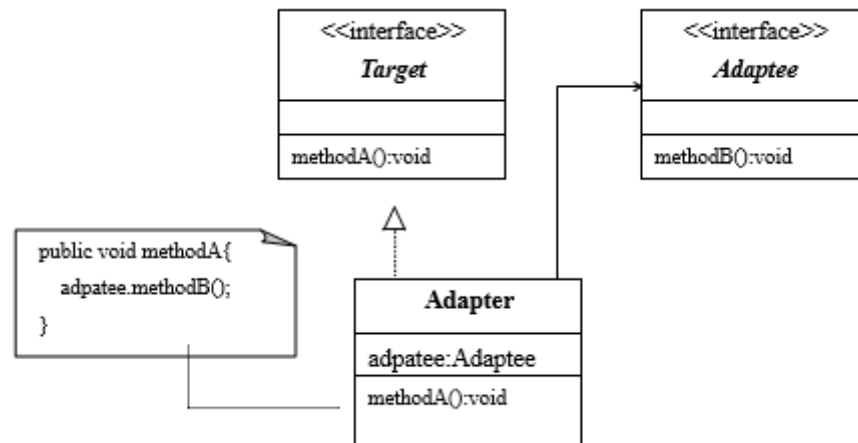
即希望一个东西可以实现已有的另一个功能

构成为：接口 - 接口 - 类

建议先实现出主体：一个 `Adapter` 实现某个接口的功能，然后内部是一个需要被适配的对象

原理

- 目标：目标是一个接口，该接口是客户想使用的接口
- 被适配者：是一个已经存在的接口或者抽象类，这个接口或抽象类需要适配
- 适配器：**适配器是一个类，实现了目标接口并包含被适配者的引用**



例题一

- 用户家里现有一台洗衣机，使用交流电，现在用户新买了一台录音机，录音机只能使用直流电。
- 由于供电系统供给用户家里是交流电，因此用户需要用适配器将交流电转化为直流电供录音机使用
 - 目标（Target）：是名字为DirectCurrent的接口
 - 被适配者（Adaptee）：是名字为AlternateCurrent的接口
 - 适配器：是名字为ElectricAdapter类，**该类实现了DirectCurrent接口并包含有AlternateCurrent接口变量。**
 - 被适配者（Adaptee）的具体类：**PowerCompany**

例题二

- 一位用户拿着业界**标准开关**StandardSwitcher，它依赖IStandardSwitchable接口才能工作。然而目前我们有个**灯**并不支持这个接口，如何实现**适配器**让标准开关支持我们的灯。

装饰模式

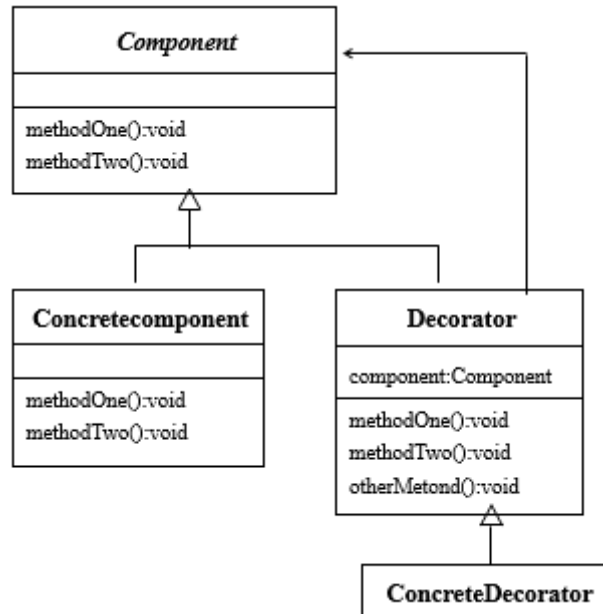
一个东西可以有很多种装饰

所有的装饰看作新的实体

抽象出其共同的抽象父类

每个装饰抽象出共同父类，**其包含被装饰者以及一个装饰作用的方法**

所有的装饰需要实现这个装饰作用的方法



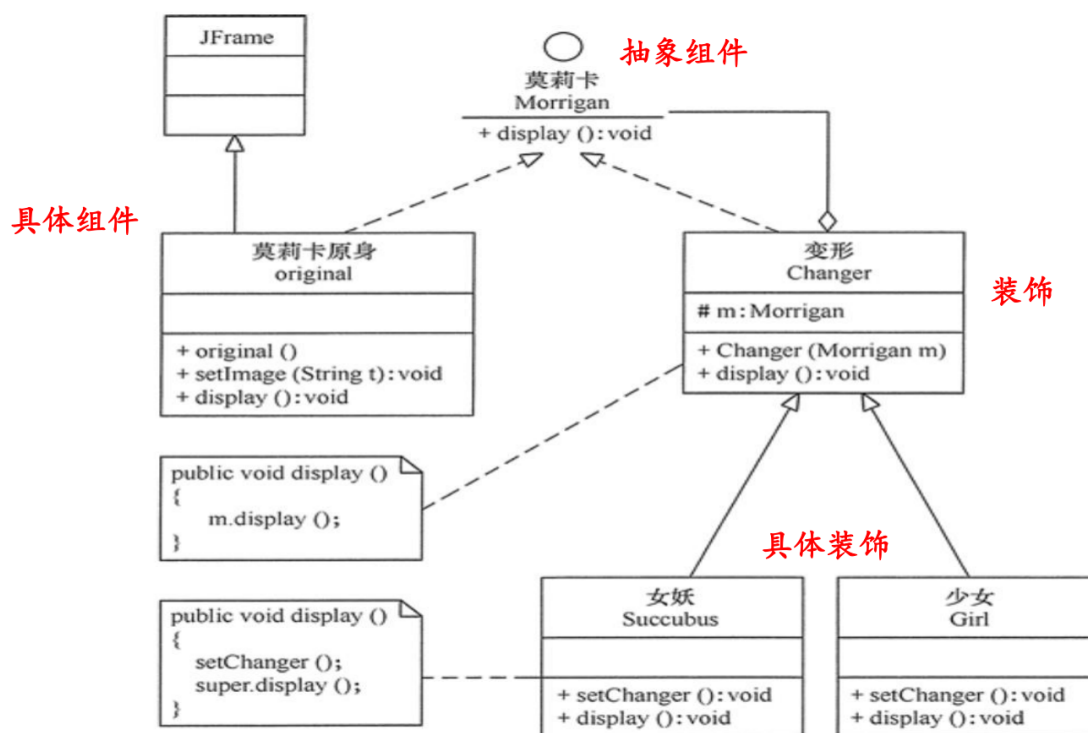
原理

- 抽象组件：抽象类，被装饰者
- 具体组件：被装饰的示例东西
- 装饰：继承抽象组件，抽象类，装饰者，作用是装饰具体组件
- 具体装饰：具体的装饰者，其也可以作为新的被装饰者

例题一

- 给麻雀安装智能电子翅膀
 - 抽象组件的名字是Bird
 - 装饰（Decorator）角色是抽象组件的一个子类，**需要包含被装饰者（抽象组件）的引用。**
 - 具体组件角色：Sparrow类，该类的实例模拟麻雀
 - 具体装饰是SparrowDecorator类，该类使用eleFly()方法去装饰fly()方法

例题二



优点

- 被装饰者和装饰着之间是**松耦合**的关系，具体装饰只知道它要装饰的对象是抽象组件的某一个子类的实例

使用场景

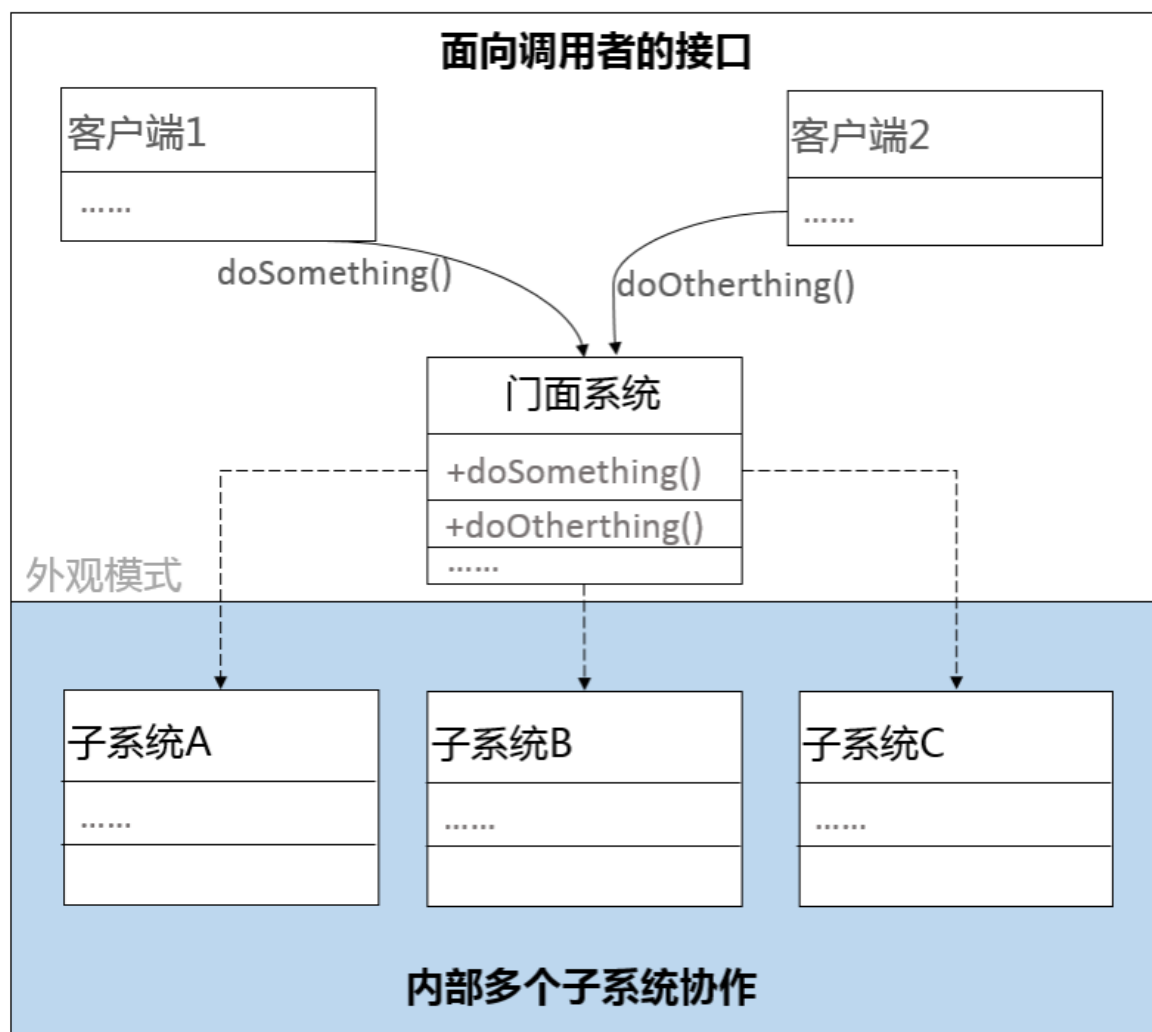
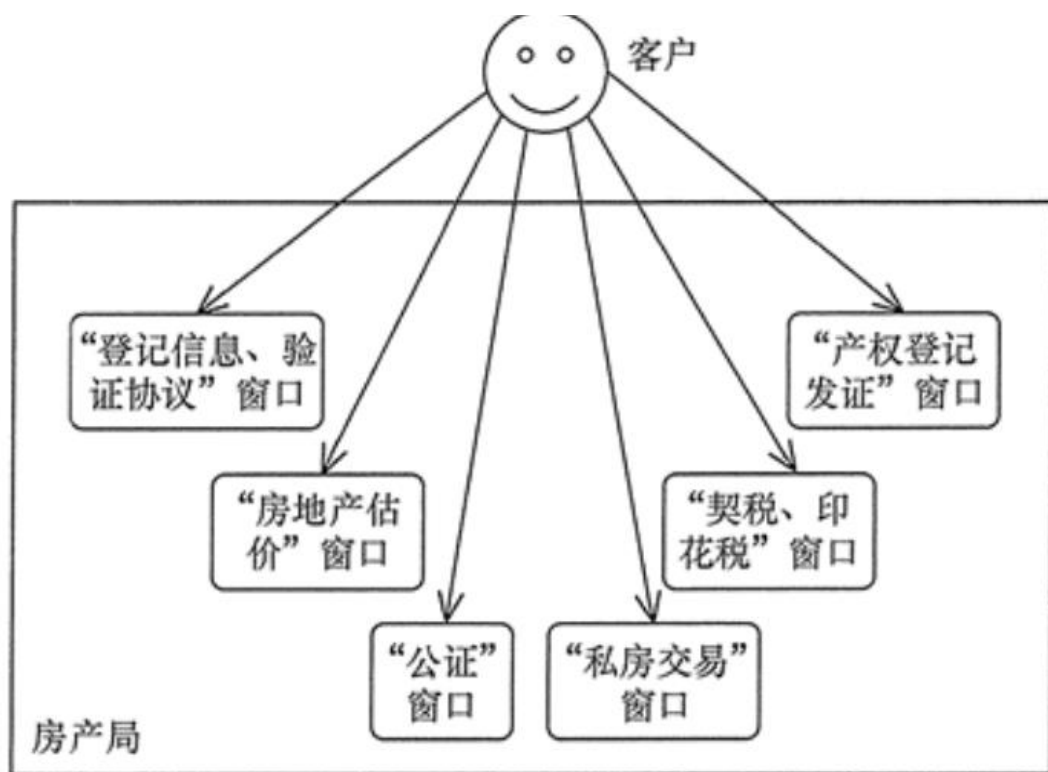
- 程序希望动态增强类的某个对象的功能
- (但采用继承来增强对象功能不利于系统的扩展和维护)

外观模式 (门面模式)

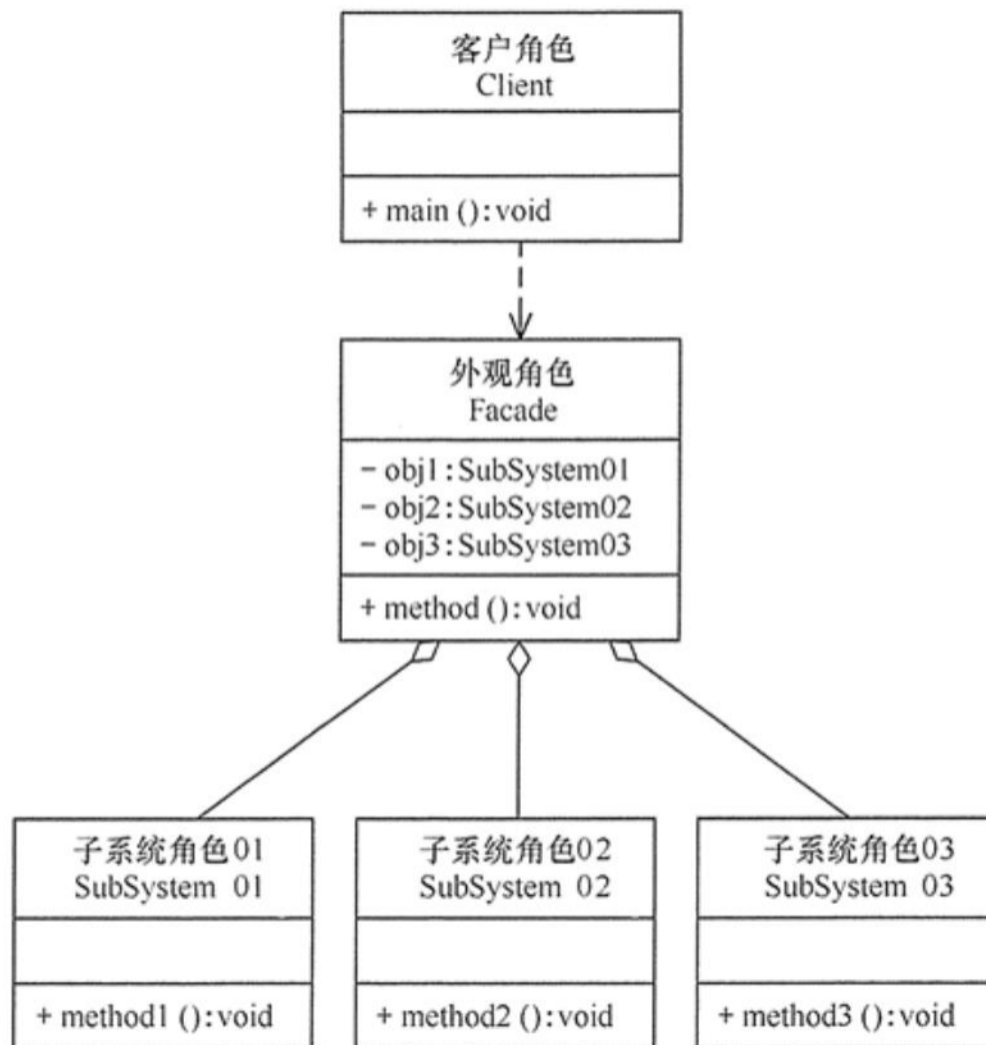
迪米特原则的典型应用

为多个复杂的子系统提供一个一致的接口，外部应用不用关心内部子系统的具体实现细节

降低应用**程序复杂度**，降低其**与子系统的耦合**，提供程序**可维护性**



例题



优点

- 降低了**子系统与客户端**之间的耦合度，使得子系统变化不影响调用它的客户类
- 对用户屏蔽了子系统组件，减少了客户处理对象的数目
- 降低了大型软件系统的编译依赖性，简化了移植过程

缺点

- 不能很好限制客户使用子系统类（外观来操作，有外观就可以直接进行子系统的任意操作，不能精确到子系统）
- 增加新的子系统可能需要修改外观类或者客户端的代码，违背了“**开闭原则**”（引入抽象给外观类，可一定程度上解决，因为对外观类的修改可以直接拓展为创建新的外观类）

行为型

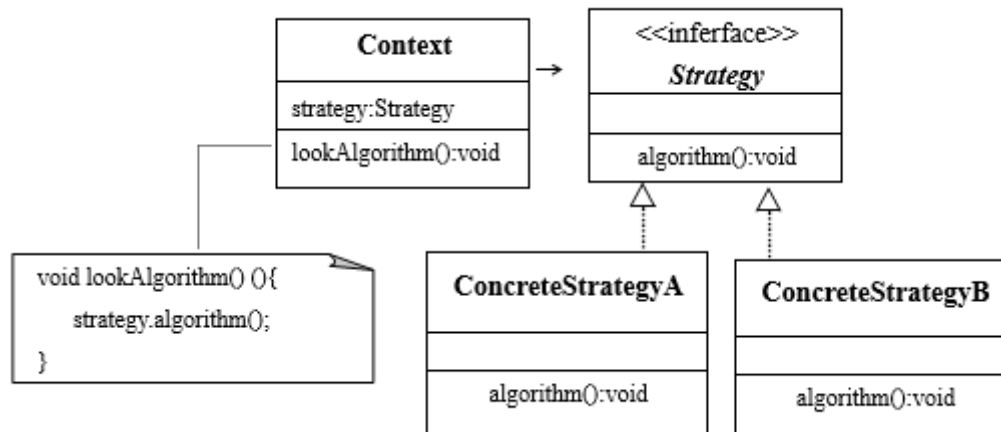
策略模式

将经常变化的部分分割出来，将每种可能的变化交给对应的**抽象类**的一个子类或实现接口的一个类区负责，从而不需要关心具体实现

一个策略接口，多个策略，一个包含策略并可设置执行策略的类，最后一个主类进行调用实现

原理

- 策略：**接口**（当然也可以为抽象类）
- 具体策略：实现策略接口的类，即具体的一些算法
- 上下文：依赖策略接口的类

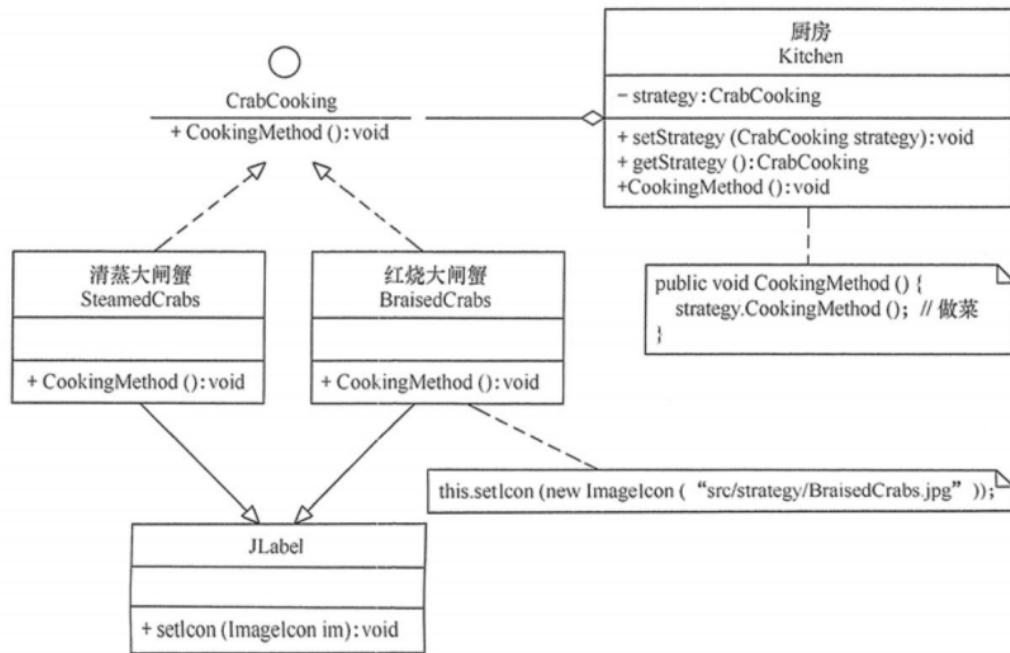


例题一

- 问题：在多个裁判负责打分的比赛中，每位裁判给选手一个得分，选手的最后得分是根据全体裁判的得分计算出来的。请给出几种计算选手得分的评分方案（策略），对于某次比赛，可以从你的方案中选择一种方案作为本次比赛的评分方案。
 - 在这里我们把策略接口命名为：**Strategy**。在具体应用中，这个角色的名字可以根据具体问题来命名。
 - 在本问题中将上下文命名为 **AverageScore**，即让 `AverageScore` 类依赖于 `Strategy` 接口。
 - 每个具体策略负责一系列算法中的一个。

例题二

家里买了两只大闸蟹，你想吃红烧的，你妈妈想吃清蒸的，想一想这个问题如何使用策略模式来实现？想一想**策略（Strategy）**，**具体策略**和**上下文（Context）**分别是什么？



优点

- 上下文和具体策略是松耦合的
- 满足“开闭原则”
- 符合面向对象的基本原则之一：少用继承，多用组合

适用场景

程序的主要类不希望暴露复杂的、与算法相关的数据结构

访问者模式

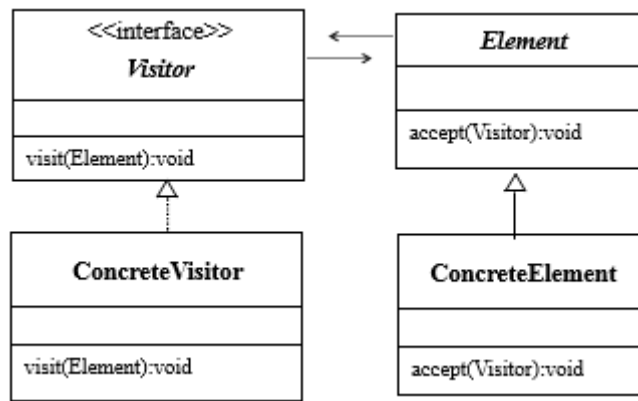
（和策略模式的实现基本相同）

相较于**策略模式**，策略模式是一个类进行使用不同的策略进行计算

访问者模式是**两个抽象类之间的相互作用**

原理

- 抽象元素，可以接收实现了访问者接口的访问者
- 具体元素
- 抽象访问者，接口，定义操作的具体元素的方法
- 具体访问者



例题一

- 问题：根据电表显示的用电量计算用户的电费。用户包括居民和企业。访问同一个电表，即分别按家用电标准和工业用电标准计算了电费。
 - 抽象的访问者：
 - 具体的访问者：居民和企业用户
 - 抽象元素：抽象类AmmeterElement
 - 具体元素：Ammeter(模拟电表)

例题二

年底，CEO和CTO开始评定员工一年的工作绩效，员工为工程师，CTO关注工程师的代码量；CEO关注的是工程师的KPI。如何使用访问者模式？抽象访问者，抽象元素，具体访问者分别指什么？

优势

- 不改变原有集合的基础上，增加对于元素的新操作，符合“开闭原则”

使用场景

- 需要对集合中的对象进行很多不同的并且不相关的操作，但我们又不想修改对象的类

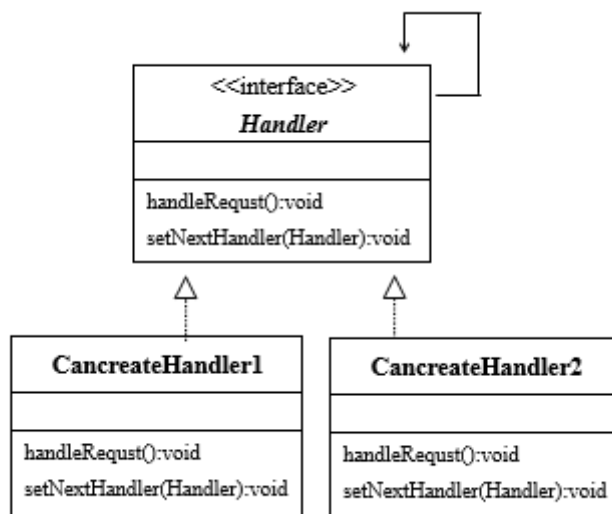
责任链模式

原理

将用户的请求分派给多个对象

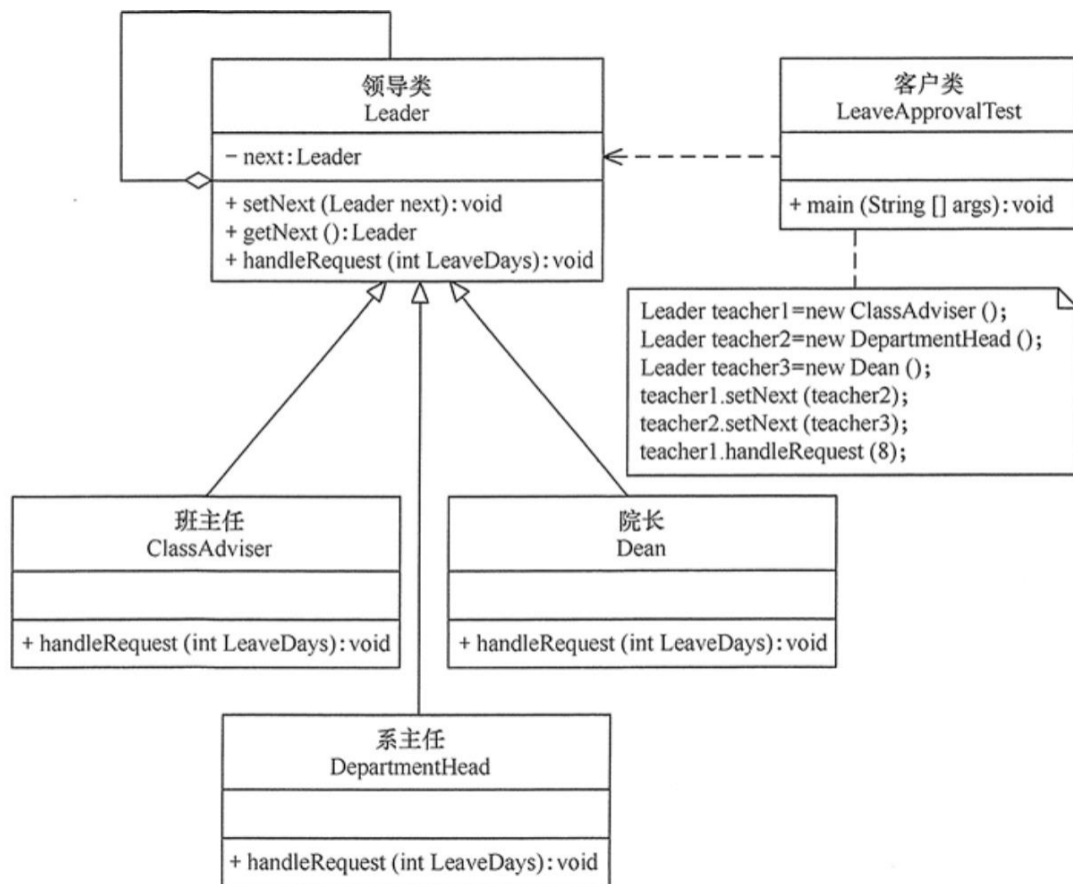
- 处理者：接口，负责规定具体处理者**处理用户请求的方法**和具体处理者设置**后继对象**
- 具体处理者：实现接口

如果在执行方法的过程中发现当前处理者可以处理用户的请求，则处理有关数据，否则就将用户的请求传递给自己的后继对象



例题一

- 假如规定学生请假小于或等于 2 天，班主任可以批准；小于或等于 7 天，系主任可以批准；小于或等于 10 天，院长可以批准；其他情况不予批准；这个实例适合使用职责链模式实现。



例题二

- 假如电影票7元,购票者购买2张电影票,给售票员100元面值钞票一张。那么实际上售票员找赎86元过程如下:
 - 首先在50元面值的钱盒中看能否完成任务,或部分任务。50元面值的钱盒发现能找赎86元中的50元(贡献一张50元的钞票),即只完成部分任务,因此把剩余的36元任务交给下一个20元面值的钱盒
 - 20元面值的钱盒发现能完成36元中的20元(贡献一张20面值的钞票),因此把剩余的16元任务交给下一个10元面值的钱盒
 - 依次类推,如果在后续某个钱盒完成了自己的找赎任务,那么售票员就完成了找赎任务,如果一直到最后一个钱盒(假设是1元面值的钱盒),该钱盒也无法完成自己的找赎任务,那么售票员就无法完成找赎
 - 使用责任链模拟售票员找赎,那么责任链上的对象就是钱盒,即责任链模式中的处理者(Handler)

优点

处理者中分配责任时,具有很高的灵活性,而不需要全是 `if - else`

使用场景

- 有许多对象可以处理用户的请求
- 程序希望动态制定可处理用户请求的对象集合

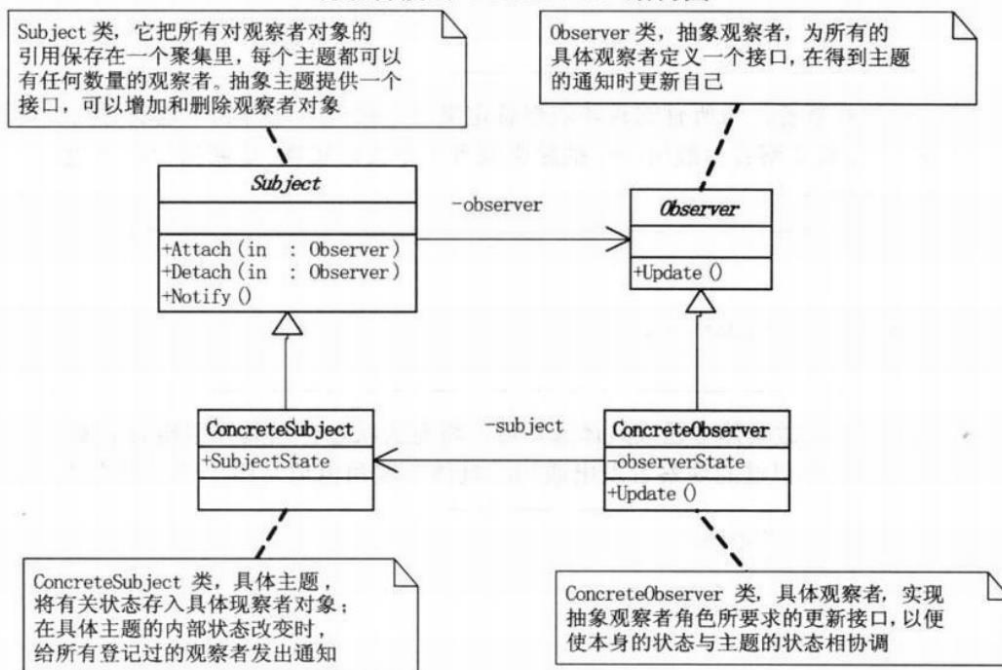
观察者模式 (监听模式)

原理

- 一个抽象主题，提供观察者的聚集类，增加和删除对象的方法，加上一个通知所有观察者的**抽象方法**
- 具体主题
- 抽象观察者，包含被通知之后更新自己的抽象方法
- 具体观察者

观察者模式定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象。这个主题对象在状态发生变化时，会通知所有观察者对象，使它们能够自动更新自己。[DP]

观察者模式（Observer）结构图



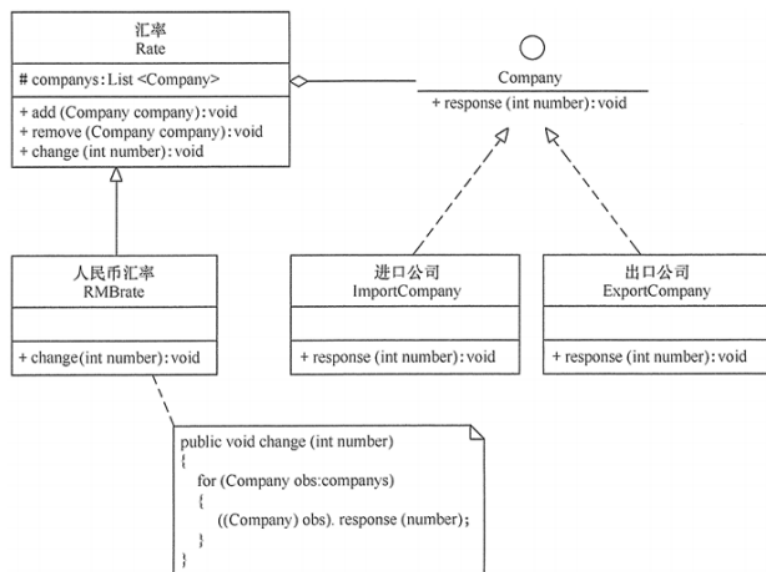
例题一

微信公众号订阅

例题二

每个不同的具体观察者都可以由不同的 `update()` 方法

- 利用观察者模式设计一个程序，分析“**人民币汇率**”的升值或贬值对**进口公司**的进口产品成本或**出口公司**的出口产品收入以及公司的利润率的影响。



使用场景

- 两个方面，一方依赖于另一方
- 一个对象的改变导致其他一个或多个对象也改变，具体的改变由被改变对象自己实现，降低对象之间耦合度
- 创建触发链，A对象影响B对象，进而影响C对象

优点

- 观察者和被观察者是抽象耦合的，松耦合的
- 可以建立一套触发机制

缺点

- 将所有的观察者都通知花费较大
- 不可以有依赖循环

基本概念

- 类和对象是抽象和具体的关系
- 面向对象的特征：封装、继承、多态
- 类与类之间的关系：泛化、实现、关联、依赖、聚集（组合、聚合）
- 静态方法与非静态方法的区别：

1. 静态方法由 `static` 修饰，可以由类名直接调用；

非静态方法只能在类被实例化成对象之后才能通过对象调用方法

2. 静态方法只能直接调用静态成员或者方法，不能调用非静态方法或者非静态成员；

非静态方法既可以调用静态成员或者方法，又可以调用其他的非静态成员或者方法

- **main()为什么要用 static 修饰：**

用static修饰的方法，**无须产生类的实例对象就可以调用该方法**。没有static修饰的方法，需要产生一个类的实例对象才可以调用该方法。main方法必须要static来修饰，是因为main方法是Java解释器调用的，那时候还没有任何对象产生

- **封装的优点：**

1. 便于使用者正确地方便地理解和使用系统，防止使用者错误修改系统的属性
2. 有助于建立各个系统之间的松耦合关系，提高系统的独立性。当某个系统的实现发生变化，只要它的接口不变，就不会影响到其它系统
3. 提高软件的可重用性，每个系统都是一个相对独立的整体，可以在多重环境中得到重用
4. 降低了构建大型系统的风险，即使整个系统不成功，个别的独立子系统有可能仍然有价值

- **封装的含义：**

1. 把对象的属性和行为看成一个密不可分的整体
2. 信息隐藏

- **单例模式的适用场合：**

1. 只需要使用一个单独的资源并需要共享器状态信息
2. 需要频繁创建和销毁对象
3. 创建对象耗时或耗资源过多，但又是常用的
4. 工具类对象
5. 频繁访问数据库或文件的对象

- **静态代码块和非静态代码块的异同点：**

- **相同点：**

1. 都是在JVM加载类时且在构造方法执行之前执行
2. 在类中可以定义多个
3. 一般在代码块中对static变量赋值

- **不同点：**

1. 静态代码块在非静态代码块之前执行
2. 静态代码块只在第一次加载时执行一次，之后不再执行，而非静态代码块在每次 new 一次就执行一次

- **继承的优缺点：**

- **优点：**

1. 避免了共用代码的重复开发，减少代码的冗余，提升代码的可复用性
2. 支持向上映射，提高程序的可扩展性
3. 继承是类实现可重用性和可扩展性的关键特征。
4. 通过继承增强一致性，从而减少模块间的接口和界面
5. 通过向上映射，支持多态，有很多好处

○ **缺点：**

1. 破坏了封装性。继承很容易改变父类实现的细节，即使父类整体没有问题，也有可能因为子类细节实现不当，破坏父类的约束

● **Java形成多态的必备条件：**

1. 必须有**继承**
2. 继承中必须有方法**覆盖**
3. 必须由父类的引用指向子类的实例，且调用被覆盖的方法，**向上转型**

● **多态的优点：**

1. 尽管子类的对象千差万别，但都可以采用**统一**的方式来调用
2. 应用程序不必为每一个子类编写功能调用，只需要对抽象基类进行处理即可，可以大大提高程序的**可复用性**
3. 派生类的功能可以被基类的引用变量引用，这叫做**向后兼容**，可以提高程序的**可扩充性和可维护性**

● **向后兼容：**可提高程序的可扩展性和可维护性，程序的调用界面清除，可读性好，并解决代码冗余

● **接口的必要性：**

1. 丰富Java面向对象的思想
2. 有利于代码的规范性
3. 有利于对代码进行维护
4. 增强安全、严密性

● **new关键字的作用：**

1. 为对象分配内存空间
2. 引起对象构造方法的调用
3. 为对象返回一个引用

● **抽象类和接口的比较：**

○ **共同点：**

1. 都不能实例化
2. 都位于系统的抽象层，都需要其他类具体实现
3. 它们都需要子类类继承或实现才有意义，最终目的是为了多态
4. 子类重写了父类的方法，再通过向上转型，由父类对象引用指向子类对象，达到运行时动态调用子类方法的目的

○ **不同点：**

1. 接口中的成员变量和方法只能是 `public` 类型的；

抽象类中的成员变量和方法可以是各级别的，抽象类比接口包含更多的实现细节

2. 接口的成员变量只能是 `public static final` 的；

抽象类可以定义各种类型的实例变量和静态变量

3. 接口中没有构造方法；

抽象类中有构造方法

4. 接口中所有方法都是抽象方法；

抽象类中可以有也可以没有抽象方法

5. 一个类只能继承一个直接的父类，这个类可以是抽象类；

一个类可以实现多个接口

6. 抽象类是某一类事物的一种抽象；

而接口不是类，它只定义了某种行为

7. 语义上，接口表示更高层次的抽象，声明对外提供的服务；

抽象类则是各种具体类型的抽象

- `instanceof` 二元运算符：

- 表达式 `obj instanceof T`，`obj` 操作数的类型必须是引用类型，否则会发生编译错误
- 如果关系表达式的 `instanceof` 产生编译时错误，则 `obj` 强制转换为 `T` 时同样发生编译错误
- 如果 `obj` 不为 `null` 并且 `(T)obj` 不抛出 `ClassCastException` 异常，则该表达式值为 `true`，否则为 `false`

- 内部类的用途：

1. 封装类型：内部类是一种封装类型的有效手段，正常类只有 `public` & `default` 访问级别，但成员内部类可以有 `public` & `protected` & `default` & `private` 四个访问级别
2. 直接访问外部类成员，内部类可以访问外部类各级别的变量方法
3. 回调外部类的方法

- 集合框架的组成：

- `Collection` - `List` & `Set` & `Queue`
- `Map`

UML

访问限制修饰符

- "+"表示 `public`
- "-"表示 `private`
- "#"表示 `protected`
- 不带符号表示 `default`

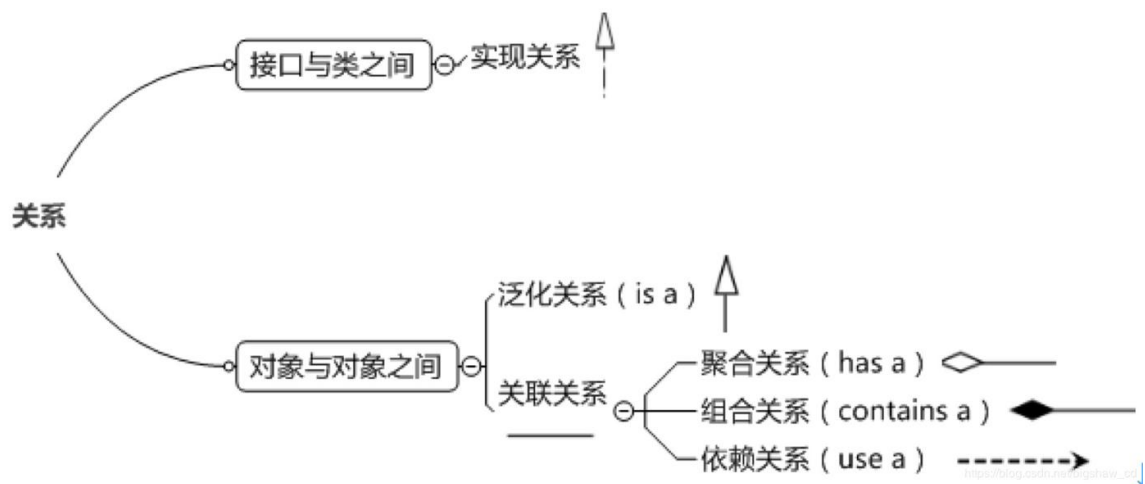
类/抽象类/接口

Java	UML
<pre>public class Hourly { private double hourlyRate; private double hoursWorked; public double computePay() { } }</pre>	

Java	UML
<pre>public abstract class Employee { private String address; private String name; private int number; public abstract double computePay() { ... } public void mailCheck() {} }</pre>	

Java	UML
<pre>public interface Shape { public double computeArea(); public double computePerimeter(); }</pre>	

类/接口之间的关系



实现

Java	UML
<pre>public interface Runnable { } public class MyThread implements Runnable { }</pre>	

继承

Java	UML
<pre>public class Employee { } public Professor extends Employee { }</pre>	

关联

Java	UML
<pre>public class Employee { private TimeCard _tc; public void maintainTimeCard() { ... } }</pre>	

依赖

Java	UML
<pre>public class Driver { public void driverCar(Car c) { ... } }</pre>	<pre>classDiagram class Driver class Car Driver ..> Car</pre>

聚合

Java	UML
<pre>public class Department { private Employee e; public Employee getEmployee() { ... } }</pre>	<pre>classDiagram class Department class Employee Department o--> Employee</pre>

组合

Java	UML
<pre>public class Employee { private TimeCard tc; public void maintainTimeCard() { ... } }</pre>	<pre>classDiagram class Employee class TimeCard Employee "1" *-- "0..*" TimeCard</pre>

Java 算法题使用方法汇总

- 标准格式输出

```
String s = String.format("%s %5d%n", user, total);
System.out.println(s);
```

- 遍历 Map 的值

```
for (Map.Entry<String, Integer> entry : entries.entrySet()) {
    System.out.println(entry.getKey() + ": " +
        entry.getValue());
}
```

- 对于 `TreeMap` 的排序，只能通过 `key` 来排序，如果想要通过 `value` 类排序，建议直接搞定 `Map` 之后转化为 `List` 排序或者使用 `PriorityQueue` 然后重载排序

- `Collections` 排序

```
import java.util.Collections
```

```
Collections.sort(list); // 对list进行升序排序
Collections.sort(list, Collections.reverseOrder()); // 对list
进行降序排序
```

```
// lambda表达式重载
```

```
List<Person> list = new ArrayList<>();
Collections.sort(list, (o1, o2) ->
o1.getName().compareTo(o2.getName()));
```

- 基本数据类型的数组排序

```
import java.util.Arrays
```

```
int[] array = {4, 3, 5, 1, 2};
Arrays.sort(array);
```

```
// 限制范围，左闭右开
```

```
Arrays.sort(numbers, 0, 4);
```

```
// lambda表达式重载
```

```
Arrays.sort(strs, (x, y) -> x.compareTo(y));
```

- 大/小根堆

```
// 小根堆
```

```
PriorityQueue<Integer> minQueue = new PriorityQueue<Integer>
();
```

```
// 大根堆
```

```
PriorityQueue<Integer> maxQueue = new PriorityQueue<Integer>
((o1, o2) -> o2 - o1);
```

```
.add(); // 添加元素
```

```
.peek(); // 取顶部元素
```

```
.poll(); // 删除顶部元素
```

- 常用集合

```
import java.util.TreeSet
```

```
import java.util.TreeMap
```

```
TreeSet
```

```
TreeMap
```

- 集合遍历

```
Iterator + iterator.hasNext() + iterator.next()
```

- 输入

```
import java.util.Scanner
```

```
Scanner in = new Scanner(System.in);
```

```
.next() || .nextLine() 方法返回值为 String
```

```
.nextInt() 返回 int
```

当然还有 `.nextDouble()` 等

- BigInteger 类使用

```
// 构造方法:
```

```
public BigInteger(String val) // 获得指定的大整数
```

```
public BigInteger(String val, int radix) // 获得指定进制的大整数
```

```
// 数据赋值
```

```
public static BigInteger valueOf(long val)
```

```
BigInteger bd5 = BigInteger.valueOf(100);
```

```
// 运算: 每次返回一个新的对象的引用
```

```
public BigInteger add(BigInteger val) //加法
```

```
public BigInteger subtract(BigInteger val) //减法
```

```
public BigInteger multiply(BigInteger val) //乘法
```

```
public BigInteger divide(BigInteger val) // 除法
```

```
public boolean equals(Object x)
```

```
// 转为整数
```

```
public int intValue(BigInteger val)
```

- StringBuilder 类使用方法

```
.append("abc") or .append(123) // 将指定字符串、字符、布尔值或其他数据类型
的表示追加到字符串缓冲区的末尾
.insert(4, "abd") // 在指定位置插入指定字符串、字符、布尔值或其他数据类型
的表示
.replace(6, 11, "abs") // 用新的字符串替换指定范围内的字符序列
.delete(6, 11) // 删除指定范围内的字符序列。
.reverse() // 反转字符串缓冲区中的字符序列
.charAt() // 获取指定索引处的字符
.substring() // 返回一个新的字符串，其中包含字符串缓冲区中指定范围内的字
符
.indexOf() // 返回指定子字符串第一次出现的索引
.length() // 返回字符串缓冲区中的字符数
```

- 字符串和数字转换：

```
// 字符串转数字
int x = Integer.parseInt("123");
double y = Double.parseDouble("1.23");
// 数字转字符串
String sx = String.valueOf(x);
String sy = String.valueOf(y);
```

- 字符串转化为小写： `String x; x = x.toLowerCase();`
- 判断一个 `char` 类型的信息： `Character.isLetter('a')`

```
isDigit(char)
isLetter(char)
isLetterOrDigit(char)
isLowerCase(char)
isTitleCase(char)
isUpperCase(char)
```

- 字符串分割 `split()`

```
temp = str.split(delimeter); // temp: String[]
```

- `List` 与 数组转化（最好还是直接循环转换）

```
// List转数组
String[] arr = list.toArray(new String[list.size()]);
// 数组转List
List<String> list = new ArrayList<>(Arrays.asList(arr));
```

- `List` 进行值修改： `list.set(index, value);`

- Map 一句话进行值插入和修改

```
mp.put(key, mp.getOrDefault(key, 0) + 1);
```

Java刷题记录

GPT

- 序列化结果一定为二进制？



- 哪些成员会被序列化？



- 什么情况可以使用函数式编程？



- 设计原则的浅显理解



- 关于protected的权限：

1. 父类的protected成员是包内可见的，并且对子类可见；
2. 若子类与父类不在同一包中，那么在子类中，子类实例可以访问其从父类继承而来的protected方法，而不能访问父类实例的protected方法。
3. 不同包下，在子类中不能通过另一个子类引用访问共同基类的protected方法
4. 对于protected修饰的静态变量，无论是否同一个包，在子类中均可直接访问
5. 对于protected修饰的静态函数，无论是否同一个包，在子类中均可直接访问。在不同包的非子类中则不可访问。

1. 同一个包中，子类对象能访问父类的 protected 方法吗？
 - ◆ 在同一个包中，普通类或者子类都可以访问基类的 protected 方法。
2. 不同包下，在子类中创建该子类对象能访问父类的 protected 方法吗？
 - ◆ 可以
3. 不同包下，在子类中创建父类对象能访问父类的 protected 方法吗？
 - ◆ 不可以
4. 不同包下，在子类中创建另一个子类的对象能访问公共父类的 protected 方法吗？
 - ◆ 不可以
5. 不同包下，子类能访问父类的 protected 静态方法吗？
 - ◆ 可以

- final修饰的变量如何被初始化

C. 被 final 修饰的变量在定义时必须初始化。

错误。被 final 修饰的变量可以延迟初始化，只要在构造函数或静态初始化块中完成初始化即可。

Java测验1

单选题

4. 以下代码输出正确的是？

```
1 public class Test {
2     public static void main(String[] args) {
3         String s1 = "nowcoder";
4         String s2 = "nowcoder";
5         System.out.println("s1 == s2 is:" + s1 == s2);
6     }
7 }
```

☐ A ture

☒ B s1==s2 is:false

☐ C s1==s2 is:true

☐ D false

单选题



7. 以下代码输出结果为：

```
1 public class IfTest{
2     public static void main(string[]args){
3         int x=3;
4         int y=1;
5         if(x=y)
6             System.out.println("不相等");
7         else
8             System.out.println("相等");
9     }
10 }
```

A 不相等

B 相等

C 第五行代码会引起编译错误

D 程序正常执行，但没有输出结果

正确答案：C 你的答案：A

官方解析：

if() 语句括号中为比较表达式，返回值要么是 true，要么是 false，if(x=y) 是将 y 赋值给 x，但是数据类型是 int 类型的，编译不能通过。

知识点：Java 工程师

Java 测验2

- 局部变量一定要初始化；成员变量不需要初始化

单选题



9. 线程结束后可以重新 start 吗？

A 可以

B 不可以

正确答案：B 你的答案：A

官方解析：

线程属于一次性消耗品，在执行完 run() 方法之后线程便会正常结束了，线程结束后便会销毁，不能再次 start，只能重新建立新的线程对象。

知识点：Java 工程师

题友讨论(6) ▲

推荐

最新

楼层

在这里输入你的观点吧



牛客948945747号 进阶牛

洛阳师范学院 Java

线程生命周期：新建，就绪，执行，阻塞，死亡。死亡后释放cpu资源，不能重复启动

👍 2 回复 发布于 2020-08-14 13:27 来自 Android 客户端



枸杞很好 飞黄腾达

中北大学 Java

run()方法: 在本线程内调用该Runnable对象的run()方法，可以重复多次调用；

start()方法: 启动一个线程，调用该Runnable对象的run()方法，不能多次启动一个线程

👍 4 回复 发布于 2019-11-16 19:19

11. 所有异常类的基类是?

- ☐ A `Java.throwable`
- ☒ B `Java.Lang.throwable`
- ☐ C `Java.Lang.Exception`
- ☐ D `Java.Lang.throwables`

Java测验3

9. 空引用可用于访问静态变量或方法?

- ☒ A 正确
- ☐ B 错误

正确答案: A

官方解析:

当我们通过一个对象的引用访问静态成员属性或者方法时, 访问操作只与所声明的引用类型相关; 与引用对象是否为 `null` 无关, 因为访问静态成员不需要实例化对象; 即便引用不为 `null`, 也与运行时多态无关, 因为静态成员是类相关的。

13. 以下选项哪一个不正确?

- ☐ A 类在使用之前, 一般需要将类实例化为对象
- ☒ B 类运行时内存中存在对象
- ☐ C 类和对象只是同一个东西的不同名称
- ☐ D 对象是一个变量, 其类型是用于声明变量的类。

正确答案: C

官方解析: 类是对象的模板, 对象是类的具体实例。

小转大可以, 大转小不行

14. 以下代码将导致:

```
1 int a1 = 5;  
2 double a2 =(float)a1;
```

- ☒ A 编译错误
- ☐ B 运行期错误
- ☐ C 没有错误
- ☐ D 运行时异常

```

public class Main {  👤 DeNeRATE-cool *
    public static void main(String[] args) {
        int a = 1;
        float b = (double)a;
        double c = (float)a;
    }
}

```

16. 基础数据类型在堆栈上分配?

A 是

B 否

在函数中定义的一些**基本类型的变量 (8种)**和**对象的引用变量**都是在函数的**栈stack**内存中分配。当在一段代码块中定义一个变量时，java就在栈中为这个变量分配内存空间，当超过变量的作用域后，java会自动释放掉为该变量分配的内存空间，该内存空间可以立刻被另作他用。

堆heap内存用于存放由new创建的对象和数组。在堆中分配的内存，由java虚拟机自动垃圾回收器来管理。在堆中产生了一个数组或者对象后，**还可以在栈中定义一个特殊的变量**，这个变量的取值等于数组或者对象在堆内存中的首地址，在栈中的这个特殊的变量就变成了数组或者对象的引用变量，以后就可以在程序中使用栈内存中的引用变量来访问堆中的数组或者对象，引用变量相当于为数组或者对象起的一个别名，或者代号。

引用变量是普通变量，定义时在栈中分配内存，引用变量在程序运行到作用域外释放。而数组 & 对象本身在堆中分配，即使程序运行到使用new产生数组和对象的语句所在地代码块之外，数组和对象本身占用的堆内存也不会被释放，数组和对象在没有引用变量指向它的时候，才变成垃圾，不能再被使用，但是仍然占着内存，在随后的一个不确定的时间被垃圾回收器释放掉。这个也是java比较占内存的主要原因，实际上，栈中的变量指向堆内存中的变量，这就是 Java 中的指针！

Java测验4

单选题

1. 以下输出是什么？

```

1  class TestIt
2  {
3      public static void main ( String[] args )
4      {
5          int[] myArray = {1, 2, 3, 4, 5};
6          ChangeIt.doIt( myArray );
7          for(int j=0; j<myArray.length; j++)
8              System.out.print( myArray[j] + " " );
9      }
10 }
11 class ChangeIt
12 {
13     static void doIt( int[] z )
14     {
15         z = null ;
16     }
17 }

```

A 1 2 3 4 5

B 什么都不会打印出来

Java测验5

- 静态方法调用非静态方法

1. 以下代码输出结果为：

```
1 class Main {
2     public static void main(String args[]) {
3         System.out.println(fun());
4     }
5
6     int fun()
7     {
8         return 20;
9     }
10 }
```

A 20

B 0

C 编译错误

• 引用的swap，无用

3. 以下代码输出结果为：

```
1 class Main {
2     public static void swap(Integer i, Integer j) {
3         Integer temp = new Integer(i);
4         i = j;
5         j = temp;
6     }
7     public static void main(String[] args) {
8         Integer i = new Integer(10);
9         Integer j = new Integer(20);
10        swap(i, j);
11        System.out.println("i = " + i + ", j = " + j);
12    }
13 }
```

A $i = 10, j = 20$

B $i = 20, j = 10$

C $i = 10, j = 10$

D $i = 20, j = 20$

• Java不允许默认值

5. 以下代码输出结果为：

```
1 class Main {
2     public static void main(String args[]) {
3         System.out.println(fun());
4     }
5     static int fun(int x = 0)
6     {
7         return x;
8     }
9 }
```

A 0

B 编程错误

正确答案：B

官方解析：Java 函数不允许参数设置默认值。

• 转换了，但是没接收传值

10. 以下代码输出结果为：

```
1 class Main
2 {
3     public static void main(String[] args)
4     {
5         String str = "nowcoder";
6         str.toUpperCase();
7         str += "www.nowcoder.com";
8         String string = str.substring(2,13);
9         string = string + str.charAt(4);
10        System.out.println(string);
11    }
12 }
```

☒ A WCODERwwwnoo

☐ B wcoderwwwnoo

☐ C WCODERwwwnoc

☐ D wcoderwwwnoc

正确答案：B 你的答案：A

官方解析：

`str.toUpperCase()` 将字符串小写字母转换为大写字母，但是它不会改变原始的字符串。`str.substring(x,y)` 返回 'x'(包含) 到 'y'(不包含) 位置的字符串。`str.charAt(x)` 返回 x 位置的字符。

知识点：Java工程师

课堂小测分析报告

- char类型的默认值是 `\u0000`，即一个unicode编码，就是空格 `\0`

D答案中的 `\0000` 直接编译错误，单纯的 `\` 后边只能加三个数字，unicode是四个

3. char 类型的属性（不包括局部变量）的默认值是下面哪一个？ [单选题]

选项	小计	比例
'\0' (答案)	62	32.29%
' '	10	5.21%
''	29	15.1%
'\0000'	69	35.94%
null	22	11.46%

正确率：32.29%

- （说法错误）子类不会无条件继承父类的无参数构造方法，但会隐式调用父类的无参数构造方法（`super()`）

另外，继承关系中，父类的super若为隐式调用，则父类构造方法就不可以仅有有参构造

6. 下列说法中哪一个是正确的？ [单选题]

选项	小计	比例
子类必须通过 <code>super</code> 关键字才能调用父类有参数的构造方法 (答案)	146	76.04%
子类必须通过 <code>this</code> 关键字才能调用父类有参数的构造方法	2	1.04%
子类无条件继承父类无参数的构造方法	32	16.67%
如果子类定义自己有参数的构造方法，就不能再调用父类的构造函数	12	6.25%

正确率：76.04%

● 垃圾回收机制无法保证在期望时间内进行内存释放

7. 关于 Java 垃圾收集，下列说法中哪一个是正确的？ [单选题]

选项	小计	比例
垃圾收集机制将检查并回收不再使用的内存 (答案)	180	93.75%
垃圾收集机制允许开发者明确指定并释放该内存	5	2.6%
程序开发者必须自己创建一个线程运行内存释放工作	1	0.52%
垃圾收集机制能在期望时间内释放被 Java 对象使用的内存	6	3.13%

正确率：93.75%

错误。

垃圾收集机制 (Garbage Collection) 无法保证在期望的时间内释放内存。垃圾收集器的运行时机和策略由 JVM 控制，程序无法直接控制何时进行垃圾回收。尽管可以通过 `System.gc()` 或 `Runtime.getRuntime().gc()` 建议垃圾收集，但它只是一个提示，垃圾收集器可能会忽略。

● Java的命令行输出：长度通过 `args.length` 获取，内部每一项从0开始编号

10. 下面的代码编译后，在命令行执行命令 `java MyProg good morning`，输出是什么？ [单选题]

```
public class MyProg {
    public static void main(String[] args){
        System.out.println(args[2]);
    }
}
```

选项	小计	比例
MyProg	1	0.52%
good	32	16.67%
morning	59	30.73%
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 2 (答案)	100	52.08%

正确率：52.08%

● `final` 方法不能被覆盖重写，故 `interface` 中不可能有 `final` 方法

18. 关于 `interface`，下列说法中哪一个是正确的？ [单选题]

选项	小计	比例
interface 中可以有 private 方法	16	8.33%
interface 中可以有 final 方法	13	6.77%
interface 可以继承其他 interface (答案)	163	84.9%

正确率：84.9%

● 运行时异常可以不管，但非运行时异常必须管

`IOException` 就属于非运行时异常，因为必须抓抛

19. 下列说法中哪一个是错误的？ [单选题]

选项	小计	比例
用户的 Java 程序可以不处理运行时异常	26	13.54%
Java 程序必须捕获或声明所有的非运行时异常	29	15.1%
Throwable 类在 java.lang 包中，是 Object 类的直接子类	29	15.1%
IOException 是运行时异常 (答案)	108	56.25%

正确率：56.25%

- 静态内部类的实例不会自动持有外部类的特定实例的引用，在创建内部类的实例时，不必创建外部类的实例。

22. 下列说法中哪一个是错误的? [单选题]

选项	小计	比例
在创建任何内部类的实例时，外部类的实例都必须已经存在 (答案)	126	65.63%
实例内部类中，可以直接访问外部类各种访问级别的所有成员，包括成员变量和成员方法。	24	12.5%
实例内部类自动持有外部类的实例的引用	16	8.33%
匿名类本身没有构造方法，但是会调用父类的构造方法	26	13.54%

正确率: 65.63%

- 24. 关于包、package 和 import，下列说法中哪一个是正确的? [单选题]

此题有误

选项	小计	比例
package 语句和 import 语句同时存在时，import 语句不能写在 package 语句之前 (+答案)	81	42.19%
类 String 可以直接使用，不需要写出 import 语句，因此它不属于任何包	2	1.04%
不是所有的 java 源文件中都必须有 package 语句，但是他们一定都属于某个包 (答案)	104	54.17%
包只在编译期有效，当编译完成后，即使将所有.class 文件放在同一目录下也不会对运行产生影响	5	2.6%

正确率: 54.17% + 42.19% = 96.36%

- Java中，int无法当作逻辑判断变量

```
// Test.java
public class Test {
    public static char a;
    static int test(int val) {
        System.out.print(String.format("<?d>", val));
        return val;
    }
    public static void main(String[] args) {
        boolean b = test(0) || test(2) || test(1);
        System.out.print(b);
    }
}
```

下列说法中正确的是哪一个? [单选题]

选项	小计	比例
这段代码无法通过编译 (答案)	68	35.42%
程序运行输出为: <?0><?2><?1>true	21	10.94%
程序运行输出为: <?0><?2>true	101	52.6%
程序运行输出为: <?0><?2>2	2	1.04%
程序运行输出为: <?0><?2><?1>3	0	0%

正确率: 35.42%

- 并不是按谁在前被匹配就走谁，而是看是恶更加贴近实际类型

1. 方法选择规则：

- Java 根据传入参数的类型选择最合适的方法。
- 如果多个方法都匹配（如重载），Java 优先选择更具体的参数类型（即更贴近实际类型的签名）。

2. 代码分析：

```
java
```

[复制代码](#)

```
fun(null);
```

- 这里 `null` 既可以匹配 `fun(String s)` 又可以匹配 `fun(Object o)`。
- Java 会选择 **更具体的类型**，即 `String`，因此输出 `s`。

```
java
```

[复制代码](#)

```
String s = null;
Object o = s;
fun(o); // fun((Object)null)
```

27. 编译下面这段代码之后运行：

```
// Test.java
public class Test {
    public static int a;
    public static void fun(String s) {
        System.out.print("s");
    }
    public static void fun(Object o) {
        System.out.print("o");
    }
    public static void main(String[] args) {
        fun(null);
        String s = null;
        Object o = s;
        fun(o); // fun((Object)null)
        fun(s); // fun((String)null)
    }
}
```

下列说法中正确的是哪一个？

[单选题]

选项	小计	比例
这段代码无法通过编译	43	22.4%
程序运行输出为 sos (答案)	50	26.04%
程序运行输出为 oos	52	27.08%
程序运行输出为 oss	4	2.08%
程序运行输出为 sss	5	2.6%
程序运行时会抛出异常 NullPointerException	38	19.79%

正确率：26.04%



- instanceof 是递归判断



- Java `char` 类型的大小是2kb;

`goto` 是保留字，但没用

31. 下列说法中哪些是错误的? [多选题]

选项	小计	比例
char 数据类型的大小为 1 字节 (答案)	123	64.06%
一个 java 源文件中必须要有 public 类 (答案)	154	80.21%
goto 是 Java 中的保留字	119	61.98%

在 Java 中，`goto` 关键字是保留字，但是不能使用。它在 Java 语言中并没有实际的功能，因此无法作为控制流语句使用。

原因

- **goto 是一个保留字**：虽然 `goto` 是 Java 的保留字之一，但并没有在 Java 语言中实现。Java 设计者选择将其保留，但并没有赋予它任何功能。
- **控制流语句**：Java 中用于控制程序执行流的语句有 `if`、`for`、`while`、`switch` 等，使用这些语句可以有效地控制程序的逻辑结构。

• Java不可使用逗号 , 连接语句

33. 阅读下面的代码:

```
public class Base {
    int x,y,z,w;
    public Base(int a,int b){
        x=a;
        y=b;
    }
    public Base(int a,int b,int c,int d){
        //assignment x=a,y=b
        z=c;
        w=d;
    }
}
```

在代码说明//assignment x=a,y=b 处写一行代码, 下列选项中哪些是正确的?

[多选题]

选项	小计	比例
Base(a,b);	93	48.44%
x=a, y=b;	71	36.98%
x=a; y=b; (答案)	186	96.88%
this(a,b); (答案)	133	69.27%

• 子类方法覆盖父类方法不可以缩小访问权限, 返回值类型只可是父类返回类型的子类或相同类 (throw的异常类型同理)

34. 已知类 A 中声明了方法 protected Object fun(int a, int b), 下列哪些方法可以在类 A 的子类中声明?

[多选题]

选项	小计	比例
public Object method (int a, int b) (答案)	133	69.27%
private Object method (int a, int b)	59	30.73%
private Object method (int a, long b) (答案)	140	72.92%
public String method (int a, int b) (答案)	112	58.33%

```
1  abstract class A { 1 个用法 1 个继承者
2      abstract protected String fun(int a, int b); 0
3  }
4  |
5      class B extends A { 0 个用法
6          public Object fun(int a, int b) { 0 个用法
7              return null;
8          }
9      }
```

• 关于成员变量和局部变量冲突情况以及变量尚未初始化赋值

41. 下列哪些注释对应的语句会引发编译错误? [多选题]

```
//Test.java
public class Test {
    public static int c;
    public static void main(String[] args) {
        int a = Test.c; // A
        int b = c; // B
        int c; // C
        int d = c; // D
        byte e = a; // E
    }
}
```

选项	小计	比例
A ✓	27	14.06%
B ✓	30	15.63%
C ✓	77	40.1%
D (答案) ✗	82	42.71%
E (答案) ✗	146	76.04%
(空)	1	0.52%

- `String` 类型对于两个相同的初始化字符串，其 `==` 也是相同的，因为内存中已经存在该字符串

42. 根据如下代码片段：

```
String s = "hello";
String t = "hello";
char[] c = {'h', 'e', 'l', 'l', 'o'};
```

下列哪些会输出 true? [多选题]

选项	小计	比例
System.out.print(s.equals(t)); (答案) ✗	185	96.35%
System.out.print(t.equals(c)); ✗	15	7.81%

System.out.print(s==t); (答案) ✓	67	34.9%
System.out.print(t.equals(new String("hello"))); (答案) ✓	176	91.67%
(空)	1	0.52%

- 即

```
String s = null;
System.out.print(s);

// 输出: null

System.out.print(null); // CE
```

`null` 是一个常量，表示“没有引用”，它可以作为对象的值，但是不能作为基本数据类型（如 `int`、`char` 等）的值。

- `System.out.print()` **方法**：它的参数可以是任何类型的对象、基本数据类型或者它们的包装类。例如，可以传递 `String`、`int`、`char`、`Integer` 等。
- **`null` 的使用**：`null` 本身是一个空引用，它不能直接传递给 `System.out.print()` 方法，因为它并不是一个具体的类型实例。在 Java 中，`null` 只能用于对象类型，而不能直接与 `System.out.print()` 方法的参数进行匹配（它期望的是一个具体的对象或基本类型）。

45. 关于方法调用 `System.out.print(null)`，下列说法中哪些是正确的？ [\[多选题\]](#)

选项	小计	比例
无法通过编译 (答案)	26	13.54%
可以通过编译	153	79.69%

调用的是 <code>public void print(String s)</code>	39	20.31%
调用的是 <code>public void print(char s[])</code>	15	7.81%
调用的是 <code>public void print(Object obj)</code>	93	48.44%
运行时输出 <code>null</code>	97	50.52%
运行时抛出 <code>NullPointerException</code>	61	31.77%
(空)	1	0.52%

- 子类加载前先加载父类，即加载父类时子类变量还没有初始化，成员变量值均为默认值

47. 编译下面这段代码之后运行：

```
// Test.java
class A {
    int n = 1;
    A() {
        System.out.println(this.n);
        System.out.println(sfun(this.fun()));
    }
    public int fun() { return this.n; }
    static int sfun(int num) { return num; }
}
class B extends A {
    int n = 10;
    B() {}
    public int fun() { return this.n; }
    static int sfun(int num) { return -num; }
}
public class Test {
    public static void main(String[] args) {
        new B();
    }
}
```

下列说法中哪些是正确的？ [多选题]



选项	小计	比例
这段代码无法通过编译	9	4.69%
程序运行输出第一行为 1 (答案)	121	63.02%
程序运行输出第一行为 0	15	7.81%
程序运行输出第一行为 10	42	21.88%
程序运行输出第二行为 1	57	29.69%
程序运行输出第二行为 10	36	18.75%
程序运行输出第二行为 0 (答案)	17	8.85%
程序运行输出第二行为-10	34	17.71%
程序运行输出第二行为-1	25	13.02%
以上说法都不对	3	1.56%
(空)	3	1.56%

- 父类加载 > 当前静态代码块加载 > 构造函数加载（优先级）


```
// Test.java
class A {
    static { System.out.print("A"); } 3
}
class A1 extends A {
    static { System.out.print("1"); } 4
}
class A2 extends A {
    static { System.out.print("2"); }
}
class B {
    static { System.out.print("B"); } 1
    static public A a;
    static A2 getA2() { return new A2(); }
    static { System.out.print("B"); } 2
}
public class Test {
    public static void main(String[] args) {
        if(B.a == null) {
            B.a = new A1();
        } else {
            B.a = B.getA2();
        }
    }
}
```

- `final` 不可以修饰构造器

- `private` 不可以修饰类

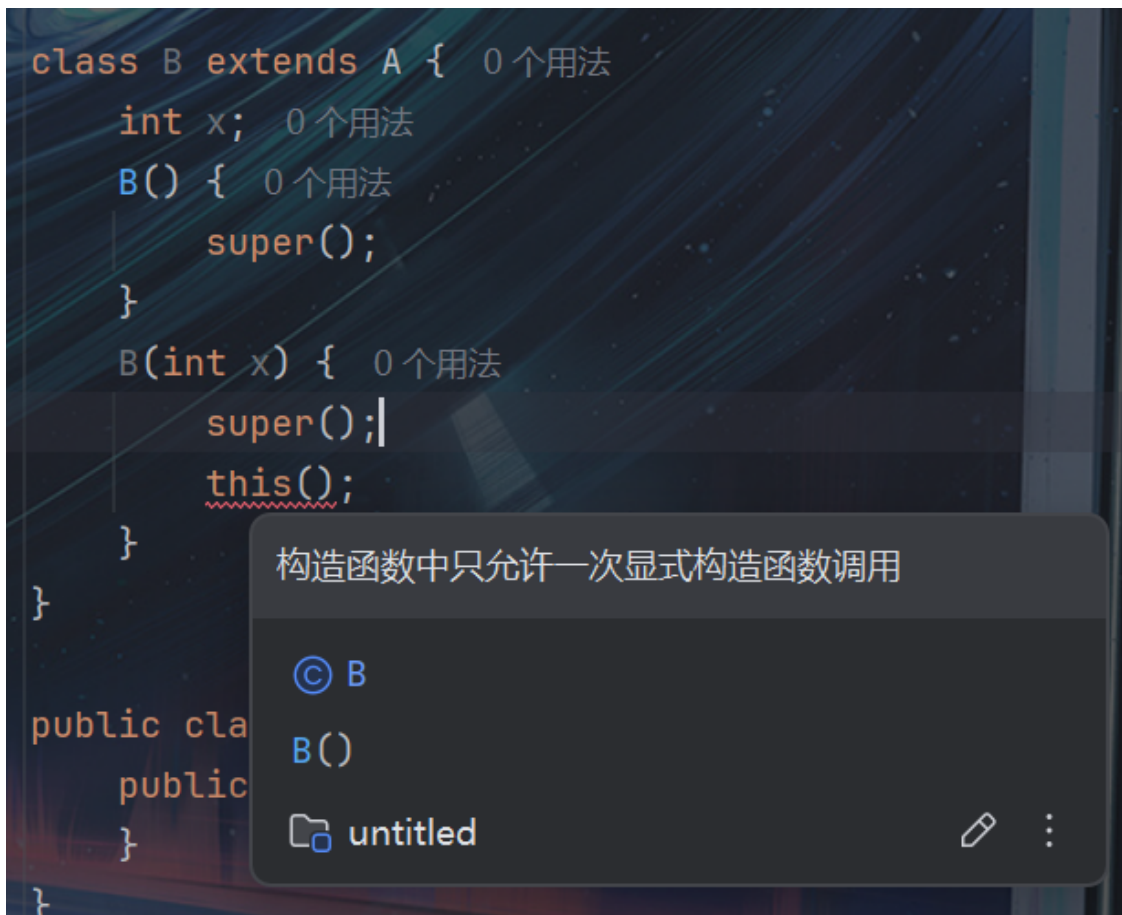
- 39. 下列哪些途径可以让类 A 无法通过 `extends` 关键字产生子类? [多选题]

选项	小计	比例
用 <code>final</code> 修饰类 A (final class A) (答案)	179	93.23%
用 <code>private</code> 修饰类 A (private class A)	66	34.38%
将类 A 所有的构造方法都用 <code>private</code> 修饰 (答案)	151	78.65%
将类 A 所有的构造方法都用 <code>final</code> 修饰	65	33.85%

2016

- 26) 在使用 `super` 和 `this` 关键字时, 以下描述正确的是_____
 - A) 在子类构造方法中使用 `super ()` 显示调用父类的构造方法, `super ()` 必须写在子类构造方法的第一行, 否则编译不通过
 - B) `super ()` 和 `this ()` 不一定要放在构造方法内第一行
 - C) `this ()` 和 `super ()` 可以同时出现在一个构造函数中
 - D) `this ()` 和 `super ()` 可以在 `static` 环境中使用, 包括 `static` 方法和 `static` 语句块





- Java的System类的 `getProperties()` 方法，方法用于获取当前系统的所有属性，区分 `getProperty()` 方法

方法签名

```
java

public static Properties getProperties()
```

 复制代码

功能

- 返回一个 `Properties` 对象，包含系统属性的键值对。
- 系统属性包括操作系统名称、版本、用户信息、Java 版本等。

常见的系统属性

以下是一些常见的系统属性的键：

键名	描述
<code>java.version</code>	Java 运行时环境版本
<code>java.vendor</code>	Java 提供商
<code>java.vendor.url</code>	Java 提供商的 URL
<code>java.home</code>	Java 安装目录
<code>java.class.path</code>	Java 类路径
<code>os.name</code>	操作系统名称
<code>os.arch</code>	操作系统架构
<code>os.version</code>	操作系统版本
<code>user.name</code>	用户的账户名称
<code>user.home</code>	用户的主目录
<code>user.dir</code>	用户的当前工作目录

示例代码

获取并打印所有系统属性

```
java

import java.util.Properties;

public class SystemPropertiesExample {
    public static void main(String[] args) {
        // 获取系统属性
        Properties properties = System.getProperties();

        // 遍历并打印所有属性
        properties.forEach((key, value) -> {
            System.out.println(key + " = " + value);
        });
    }
}
```

 复制代码

```
}
```

获取指定属性

java

 复制代码

```
public class SpecificPropertyExample {
    public static void main(String[] args) {
        // 获取操作系统名称
        String osName = System.getProperty("os.name");
        System.out.println("操作系统: " + osName);

        // 获取用户的工作目录
        String userDir = System.getProperty("user.dir");
        System.out.println("当前工作目录: " + userDir);

        // 获取 Java 版本
        String javaVersion = System.getProperty("java.version");
        System.out.println("Java 版本: " + javaVersion);
    }
}
```

注意事项

1. 安全性:

- 某些属性可能涉及系统安全信息（例如用户目录），在安全管理器启用时，可能会抛出 `SecurityException`。

2. 修改系统属性:

- 可以使用 `System.setProperty(String key, String value)` 修改某些系统属性，但不能修改所有属性，例如 `os.name` 是只读的。

3. 属性来源:

- 系统属性可以通过命令行参数 `-DpropertyName=value` 设置。
- 示例：运行程序时添加 `-Dapp.config=custom`，然后使用 `System.getProperty("app.config")` 获取值。

输出示例

执行打印所有系统属性的代码可能会输出如下内容（不同环境略有差异）：

lua

 复制代码

```
java.version = 17.0.8
java.vendor = Oracle Corporation
os.name = Windows 10
os.arch = amd64
os.version = 10.0
user.name = Admin
user.home = C:\Users\Admin
user.dir = C:\MyProject
file.separator = \
```

通过 `System.getProperties()`，可以全面了解运行环境信息，方便调试和配置管理。

🔊 📄 🌟 🔒 ↺

- JDK & JRE & JVM的关系

故JDK包含JRE，JRE包含JVM

- **JVM (Java Virtual Machine)**

- Java 虚拟机，是运行 Java 字节码的核心组件。它负责将字节码（`.class` 文件）翻译成机器码并执行，是 Java 程序跨平台特性的核心。

- **JRE (Java Runtime Environment)**

- Java 运行时环境，包含运行 Java 程序所需的 JVM 和核心类库（如 `java.lang`、`java.util` 等）。JRE 是 JVM 的运行环境，用户只需安装 JRE 即可运行 Java 应用程序。

- **JDK (Java Development Kit)**

- Java 开发工具包，提供了开发 Java 程序所需的工具和资源，包括 JRE、编译器（`javac`）、调试工具（`jdb`）、文档生成工具（`javadoc`）等。JDK 是 Java 开发者的必备工具。

- try-catch-finally语句的逻辑

解析

在 Java 的 `try-catch-finally` 结构中，无论 `try` 或 `catch` 中是否有 `return` 语句，`finally` 块的内容总会执行（除非程序在 `try` 或 `catch` 中因调用 `System.exit(0)` 直接退出）。

执行顺序

1. `try` 块中的代码首先执行。
2. 如果 `try` 中有 `return`，会先暂存返回值，但不会立即返回，而是先执行 `finally` 块。
3. 如果 `finally` 中没有修改返回值，则返回 `try` 或 `catch` 中的值。
4. 如果 `finally` 中修改了返回值（通过重新赋值），则最终返回的是 `finally` 块中的值。

- 直接赋值String变量和new String()区别

`String s = new String("abc");` 和 `String s = "abc";` 的主要区别在于 **创建对象的方式** 和 **字符串存储的位置**。下面详细说明两者的不同：

1. `String s = "abc";`

这种方式直接使用 **字符串字面量**，在编译时就会将 `"abc"` 存储到 **字符串常量池** 中。

- 如果字符串常量池中已经有 `"abc"`，`s` 会直接引用池中已有的字符串对象。
- 如果字符串常量池中没有 `"abc"`，会在池中创建一个新的字符串对象，然后引用它。

这种方式效率较高，因为避免了创建新的堆对象。

2. `String s = new String("abc");`

这种方式显式地使用 `new` 创建了一个字符串对象。

- 它会在 **堆内存** 中创建一个新的字符串对象，即使 `"abc"` 已经存在于字符串常量池中。
- 如果 `"abc"` 不在字符串常量池中，它也会先将 `"abc"` 加入到字符串常量池中，但 `s` 引用的对象是堆内存中新创建的对象。

这种方式会显式地分配堆内存，效率比直接使用字符串字面量低。

`true false false true true true`

```
public class Test1 {
    public static void main(String[] args) {
        String s1=new String("abc");
        String s2=s1;
        String s3="abc";
        System.out.println(s1==s2);
        System.out.println(s2==s3);
        System.out.println(s1==s3);
        System.out.println(s1.equals(s2));
        System.out.println(s1.equals(s3));
        System.out.println(s2.equals(s3));
    }
}
```

- 抽象类可以有构造函数
接口没有构造函数